

The essential components of software architecture design and analysis

Rick Kazman *, Len Bass, Mark Klein

Software Engineering Institute, University of Hawaii and Carnegie Mellon University, 4500 Fifth Ave., Pittsburgh, PA 15213, United States

Received 6 November 2005; received in revised form 29 April 2006; accepted 1 May 2006

Available online 27 June 2006

Abstract

Architecture analysis and design methods such as ATAM, QAW, ADD and CBAM have enjoyed modest success and are being adopted by many companies as part of their standard software development processes. They are used in the lifecycle, as a means of understanding business goals and stakeholders concerns, mapping these onto an architectural representation, and assessing the risks associated with this mapping. These methods have evolved a set of shared component techniques. In this paper we show how these techniques can be combined in countless ways to create needs-specific methods in an agile way. We demonstrate the generality of these techniques by describing a new architecture improvement method called APTIA (Analytic Principles and Tools for the Improvement of Architectures). APTIA almost entirely reuses pre-existing techniques but in a new combination, with new goals and results. We exemplify APTIA's use in improving the architecture of a commercial information system.

© 2006 Elsevier Inc. All rights reserved.

Keywords: Software architecture; Analysis methodologies; Design methodologies

1. Introduction

The importance of the right software architecture to a development effort is widely recognized. This claim is probably not controversial. Consequently this might be an odd time and place to ask “why”. Why is software architecture a critical software artifact? The simple answer is, software architecture is important *by definition*. Software architecture was *invented* to be an artifact:

- defined in terms of elements whose grain is *coarse* enough so that the design of relatively large systems can be represented in a human-comprehensible form and consequently could serve as a communication vehicle; and
- whose specification is *detailed* enough to reason about relative to the satisfaction of critical system requirements.

Based on this simple observation, we can state a core principle of software architecture:

Principle 1: A software architecture should be defined in terms of elements that are coarse enough for human intellectual control and specific enough for meaningful reasoning.

Principle 1 alone is not, however, sufficient to reap the potential benefits of software architecture. Principle 1 helps to make the software architecture right. Without understanding in addition how to “make the right software architecture”, clearly we fall short.

At the Software Engineering Institute (SEI) we have been working on software architecture-related methods and tools for over a decade and have considerable experience in applying architecture-based design and analysis methods to real systems from a wide variety of domains. From this experience base we have come to believe that there are two other essential principles for realizing the benefits of software architecture.

* Corresponding author. Tel.: +1 412 2681588; fax: +1 412 2685758.

E-mail addresses: kazman@sei.cmu.edu (R. Kazman), ljb@sei.cmu.edu (L. Bass), mk@sei.cmu.edu (M. Klein).

Principle 2: Business (and/or mission) goals determine quality attribute requirements.

Principle 3: Quality attribute requirements guide the design and analysis of software architectures.

A software architecture lives at the fulcrum between a system's business/mission goals and its implementation. Principle 2 captures the point that business goals provide the *raison d'être* for the system. Business goals lead to quality attribute goals which, as stated by Principle 3, provide analytic underpinnings for why an architecture should exhibit one type of design versus another.

Together, these three principles allow us to understand why software architecture is important, what purpose it is intended to fulfill, and whether it in fact fulfills that purpose. These principles have been realized in several of our analysis and design methods: the SAAM (Software Architecture Analysis Method) (Kazman et al., 1994), the QAW (Quality Attribute Workshop) (Barbacci et al., 2003), the ATAM (Architecture Tradeoff Analysis Method) (Kazman et al., 1999), the ADD (Attribute Driven Design) method (Clements et al., 2002), and the CBAM (Cost Benefit Analysis Method) (Kazman et al., 2001).

More recently, however, we have begun to explore the component *techniques* that link these methods to the above principles. By thinking in terms “components” we can combine these techniques in new ways and create new methods tailored for specific contexts. In this paper we enumerate these techniques and describe how they led to the creation of just such a tailored method, called APTIA (Analytic Principles and Tools for the Improvement of Architectures). We illustrate the use of APTIA on a large commercial system. While we believe that APTIA solves problems not solved by existing methods, the point of this paper is not so much to introduce APTIA as to show the generality of the techniques behind APTIA, and indeed behind all of our methods—to show how they are related to the principles described above, and to show how such techniques can be combined in limitless ways to create methods that are tailored to specific needs.

2. Techniques used in architecture analysis

Architecture analysis methods have enjoyed modest success in the software engineering practitioner community. Such methods are included in documented software engineering life-cycles, and dovetail nicely with commonly used software processes, such as RUP and XP (Kazman et al., 2004; Nord et al., 2004). In addition, architecture analysis methods are being taught in university and professional development courses on a regular basis. Typically these methods analyze an architecture for some form of modifiability, maintainability, or reusability (SAAM (Kazman et al., 1994), SAAMER (Software Architecture Analysis Method for Evolvability and Reusability) (Lung et al., 1997), ALPSM (Architecture Level Prediction of Software

Maintenance) (Bengtsson and Bosch, 1999), ALMA (Architecture-level Modifiability Analysis) (Lassing et al., 2002), or multiple quality attributes (ATAM (Kazman et al., 1999), SBAR (Scenario-Based Architecture Reengineering) (Bengtsson and Bosch, 1998), SAEM (Software Architecture Evaluation Model) (Duenas et al., 1998). A number of papers have presented a critical comparison of such methods (Ali Babar et al., 2004; Dobrica and Niemela, 2002; Kazman et al., 2005), which is an indication that this field is relatively mature and rich.

The SEI family of analysis and design methods, and indeed most architecture analysis and design methods, work by explicitly identifying the system's business goals or context, capturing evaluation criteria as scenarios, choosing among criteria based on active stakeholder participation, and having architects explain how the architecture satisfies the criteria.

The SEI methods use a number of techniques in common:

1. The explicit *elicitation of business goals*. Our methods all require a presentation of business goals. This enables evaluators to determine the criteria with which to evaluate a system. The output of the methods interprets architectural decisions in light of their impact on the business goals. This provides a means for communicating to management the business impact of technical decisions.
2. Active *stakeholder participation and communication*. Stakeholder concerns are not always expressed in documents and are not always well understood by the development team. We include stakeholders in our methods and ensure that they participate in the prioritization of business goals and in setting the focus of the methods. We have developed techniques, such as the “utility tree” (Clements et al., 2002), to aid in structured scenario elicitation and prioritization.
3. The explicit *elicitation of architecture documentation and rationale* in standardized views (Clements et al., 2003). To evaluate an architecture it is necessary for the architecture to be unambiguously represented and clearly understood. Since software architecture is, as stated in Principle 1, defined to be at the level of granularity that enables human comprehension, we require that there be such a representation.
4. The use of *quality attribute scenarios* to characterize stakeholder concerns. Business goals exist at different levels of abstraction. To evaluate a design, business goals must be expressed in terms that are operational for the architects. As stated in Principle 2, business/mission goals determine the quality attribute requirements. Quality attribute requirements must be expressed in a fashion that is unambiguous. We use a representation of quality attributes called 6-part scenarios to express the realization of business goals, and we use “general scenarios” to aid in the elicitation of 6-part scenarios (Bass et al., 2003).

5. The *mapping of quality attribute scenarios onto the architecture representation* to determine the focus of the architecture analysis. Even though architectures are defined to be understandable, they still represent large systems and contain much detail. The scenarios are used to focus on high priority aspects of the architecture.
6. The representation of *design primitives, called tactics* (Bachmann et al., 2003), to make the process of design more consistent and to explicitly link design operations to desired quality attribute goals.
7. The use of *templates to capture information* and make the methods more consistent across evaluators. Consistency in the execution of a method can only be achieved if there are templates for recording the elicited information and the analyses generated. This provides a repeatability to the gathering and documenting of information that is useful both for the evaluator and the consumer of the evaluation.
8. The explicit *elicitation of costs and benefits* associated with architectural decisions (Kazman et al., 2001). To rank and make architecture improvement decisions it is necessary to elicit information about costs, benefits, and schedule implications of architectural decision, since these concerns always “trade off” with pure quality attribute concerns.

These techniques facilitate the location of the part of an architecture to examine to determine whether a particular scenario may be achieved. But these techniques provide little support in determining *what* to examine at that location. Most architecture-based methods, ours included, rely heavily on the expertise of designers and analysts when examining the architecture. Expert opinion has typically been required to determine if the architectures are satisfactory or not.

To address this shortcoming we have recently added two additional techniques to the list above and our new method exploits these techniques:

9. Architectures can be analyzed through the use of *quality attribute models*. Some quality attributes such as performance have well established analysis models (Klein et al., 1993). Other quality attributes such as variability, testability, and security have less mature models. We use these models to help understand the design decisions made in the architecture.
10. Quality attribute models lead to a set of *quality attribute design principles*. Given a particular problem identified by the analysis, there needs to be some way to generate alternatives for improvement. We have identified a set of design principles based on quality attribute models that assist in identifying alternatives.

We now turn to an example that demonstrates the generality and composability of the above techniques (other examinations of their generality may be found in Kazman et al. (2004); Nord et al. (2004)).

3. APTIA

Our new method is named APTIA (Analytic Principles and Tools for the Improvement of Architectures). The goal of APTIA was to extend the existing architecture analysis framework in both breadth and depth. *Depth* refers to the fact we do more detailed analyses in APTIA than in our other methods and *breadth* refers to the fact that the output of APTIA is design alternatives that will improve the architecture. Since APTIA is relatively new, we are not focusing on the steps of the method as the primary point of interest, but rather on how our pre-existing techniques were re-used, and how our new techniques interact with the existing techniques. The point of this paper, and the point of the APTIA example, is to show that architecture design and analysis can be broken down into a number of modular steps, each of which have well-defined and time-tested techniques that can be successfully reused. Although the APTIA method is a new method, most of its component techniques are *not* new, and this made its application simple and, in the end, successful. This is, in fact, the hallmark of a successful engineering discipline—that its component techniques (what Shaw calls *unit operations* (Shaw, 1990)), are well understood, documented, and reused.

Our description of the example system has been intentionally obscured to protect the interests of the company, but the essence of the analysis is unaffected. The system is a three million line, interactive, real-time information system, built on a product line architecture. This system was being simultaneously marketed to several large integrator/resellers, and, as a consequence, it had a large number of real-time performance, availability, modifiability, and variability requirements.

3.1. Conceptual flow

The phases of the APTIA analysis that we performed on the target system were the following:

- Perform an ATAM.
- Determine the focus for analysis based on risk themes identified in the ATAM.
- Use quality attribute models related to the risk themes to understand the architecture.
- Use insights gained from model-based analysis and design principles to propose alternatives.
- Rank the alternatives based on costs/benefits.
- Make design decisions.

The first phase of APTIA involved performing an ATAM on the target system. The output of an ATAM is a collection of quality attribute scenarios, associated risks, and risk themes related to the business goals (Clements et al., 2002). The phase of the analysis *beyond* the ATAM involved having the system architects and managers determine which scenarios and business goals they wished to focus on. They chose to focus the APTIA analyses on

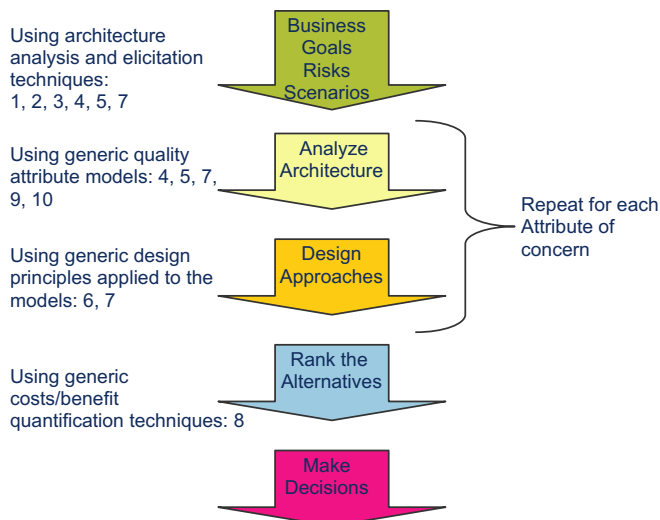


Fig. 1. APTIA's conceptual flow.

real-time performance, availability, modifiability, and variability. To illustrate the method, we first provide detail about our activities with respect to a real-time performance risk theme. We will then present an analysis of a variability risk within the architecture. We performed similar activities for other quality attributes as well.

The conceptual flow of the entire APTIA process, annotated with the techniques used, is visualized in Fig. 1.

3.2. Templates

As described above, consistency in the execution of a method is best achieved if there are templates for recording the elicited information and the analyses generated (technique 7). In this spirit, we created two templates for docu-

menting the outputs of APTIA: one for analysis and one for architectural alternatives.

The template for *analysis* is given in Table 1.

The template for capturing an *architectural alternative* is shown in Table 2.

In executing APTIA we filled out many of the analysis templates remotely, interacting with the architecture team via email and teleconferences. We would propose a hypothesis based on our understanding of the architecture, have that corrected by the architecture team, and then propose another hypothesis. This phase of iterations took approximately three weeks of part-time effort on the part of the evaluators and the architecture team.

4. An example analysis for performance

We exemplify our analysis of the real-time performance risk theme by the following scenario:

User selects next item in menu. System highlights the selection within 20 ms.

The users of the system had complained that the system would occasionally exhibit long response times. This risk theme caused customer unhappiness, system returns, and damage to the reputation of the manufacturer, which clearly affected important business goals. This allowed us to fill out parts 1 and 2 of the analysis template. Part 3 is described next.

4.1. Detailed analysis of the architecture

One of the limitations of our earlier methods, such as the ATAM, is that there is seldom time to do more than

Table 1
APTIA template for architecture analysis

Template component	Rationale
1 A scenario of interest	The analysis is based on and initiated by a scenario coming from an ATAM, or perhaps some other source. This scenario identifies the focus of the analysis
2 All associated risk themes and business goals	The risk themes and business goals justify why this scenario is important
3 A detailed quantitative or qualitative analysis of the architecture	This is the heart of the template. It contains analysis as to why there is a problem in achieving the scenario's response goal
4 Relevant items for stakeholder communication	Although a template captures the analysis results the stakeholders need an explanation of the rationale for the analysis and how it leads to the risks

Table 2
APTIA template for architecture alternatives

Template component	Rationale
1 A reference to the appropriate analysis model	This is a pointer to the Template for Architecture Analysis (Table 1)
2 A description of the architecture alternative	The portions of the architecture that would be modified are described
3 A rationale for the alternative	This section couples the alternative to the analysis model. A change in the architecture will affect the model and the new model should have a better response with respect to the quality attribute scenario
4 A discussion of the impact to the business goals, including an analysis of costs and benefits	Changes to the architecture will engender costs and will have benefits with respect to the quality attributes and hence to the business goals

a superficial analysis of any scenario. Spending even a single hour on a scenario is a significant portion of an ATAM. This is appropriate given the goals of the ATAM—to elicit architecturally relevant information, to focus stakeholder attention on architecturally relevant issues, and to understand the broad sweep of the architecture and its relationship to its business goals—but it seldom provides a sufficient basis to *deeply* understand an architecture. Furthermore, an ATAM *never* has the time to create a detailed qualitative or quantitative model of any architectural approach. During the creation of APTIA’s analysis templates, we had more time to think about portions of the architecture than is possible during an ATAM.

Since the scenario highlighted a performance issue, we looked at a concurrency view of the architecture. This led us to focus on the arrival rates and distributions of messages, how processing resources were allocated and prioritized, what shared resources existed, and how these were arbitrated. During the ATAM we had collected, among the other architectural documentation, a concurrent thread diagram. We reviewed this diagram with the architecture team and annotated it with detailed information that had not been provided during the ATAM, such as the priorities of all the threads. The resulting annotated diagram is shown in Fig. 2.

Since this system uses fixed priority scheduling we can use the performance principles (technique 10) associated with fixed priority scheduling to guide our analysis. These principles are derived from the theory of Rate Monotonic Analysis (Klein et al., 1993):

- Priorities should reflect deadlines. The theory tells us that executions with shorter deadlines should be given higher priority in the assignment of the processor.
- FIFO queues are problematic. FIFO queues do not reflect the deadlines associated with the tasks in the queue.

- Look for sources of preemption and blocking. These may prevent higher priority tasks from proceeding. Potential sources of blocking are
 - Non-preemptable computations.
 - Critical sections.
 - Critical instance.
 - Shared or critical resources (e.g. sizes/capacities of memory, thread pool, queues, DVD drives, bus connections, ...).
 - Execution time. A long execution time at a high priority will have the effect of preemption.
 - Arrival rates. A high arrival rate of a high priority task will preempt progress on lower priority tasks.
- Identify end-to-end and intermediate deadlines and the priorities of threads visited along the way. Computation in response to a stimulus may involve multiple threads and multiple priorities. Intermediate deadlines justify giving a particular section a higher priority.

These principles were enumerated prior to the on-site visit. They depend only on the quality attribute model and hence are applicable for any system and not just the system being evaluated. The most important principle for the scenario under consideration was that the priority accorded to a task should be consistent with the task’s deadline: the shorter the deadline, the higher the priority. Given this principle, we examined the architectural properties that can compromise a task’s priority, as identified above. For example, if many messages with different deadlines arrive at a FIFO queue, messages that are destined to be processed by higher priority tasks can “get stuck” behind messages destined to be processed by lower priority tasks. A thread pool that is too small might also compromise the achievement of a deadline.

Given this basis of understanding, drawn from the set of performance principles exemplified above, we now had

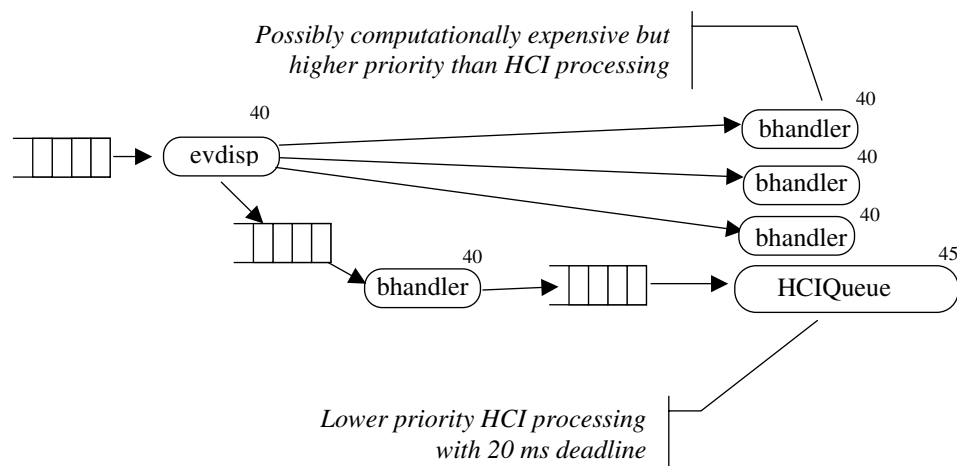


Fig. 2. Concurrent thread diagram.

enough information to create the third part of the analysis template:

4.1.1. Analysis assumptions

- All queues are FIFO.
- The dispatcher thread (evdisp) dispatches both short running HCI events and long running application events to behavior handlers (bhandler).
- Limited size of the thread pool (a critical resource).

4.1.2. Analysis conclusions

Based on the collected information, we identified several situations that could result in missing the 20 ms deadline specified in the scenario:

- There were two types of events event in the initial FIFO queue: graphics events with relatively short deadlines and application events with relatively long deadlines. The first problem is that graphic commands can “get stuck” behind any sequence of events including application events.
- The second problem is that long-running application events can preempt short running graphics commands in the lower priority HCIQueue thread for long periods of time.

It is important to note at this point that what we have constructed and documented here is a “qualitative proof” of a potential problem within the architecture. This is different than the ATAM (or ALMA, or ALPSM, or SAA-MER, or any other intervention-based architecture analysis method), where such “proofs” are a luxury that time does not permit. In part this is because these the methods were never intended to create such proofs—they are all focused on mapping scenarios onto an architecture as a means of illuminating potential risks.

4.2. Relevant items for stakeholder communication

Lastly, the analysis template contains a section for stakeholder communication. During the entire APTIA process the “conversation” between the analysts, architectures, and other important stakeholders is recorded. This becomes a repository of design rationale. So, for example, the analysis template would record rationale such as

Question: What is the size of the thread pool? Is the size of the thread pool ever too small to accommodate all outstanding events?

Answer: The thread pool will be dynamically resizable and limited by the available memory.

Question: Can we raise the priority of the HCIQueue thread?

Answer: It can be raised to be higher than the application event handler threads, but it cannot be raised too high as this might result in screen flicker.

4.3. Architecture alternatives

Once architectural problems have been identified by APTIA, the next step is to determine a set of potential improvements to address these problems. The generation of architectural alternatives was performed in a workshop where the evaluators and the developers were co-located. The co-location made it easier for the architecture team to eliminate unworkable alternatives. Co-location also facilitated communication of the principles that led to the generation of the alternatives. These principles were based on quality attribute models, as presented in Section 4.1 above, and on design tactics (technique 6). Now we turn to the template that enumerates the design alternatives.

4.4. Reference to the appropriate analysis model

Part 1 of the architecture alternatives template is a pointer to the analysis that had been performed describing the problem, as exemplified in Section 4.1.

4.5. Describing architecture alternatives

While APTIA shares most of its techniques with its predecessors, its biggest difference is that it includes *design* as portion of the method. While this is not a full-blown architecture design method, it recognizes that designing a solution to a problem is easier when the problem has been precisely characterized and analyzed. The performance analysis model built in Phase 1 provides a ready-made proving ground in which to “test-drive” any proposed changes to the architecture. In this way, proposed alternatives can be analyzed and compared to each other in a systematic way. These are captured in part 2 of the documentation template.

For each problem identified in the analysis, we create architectural alternatives (potential design solutions), based upon documented design primitives: tactics (Ali Babar et al., 2004). For each quality attribute of interest, we identified principles to guide the discussion and to educate the architecture team. In the case of our performance problems, we identified two tactics as worthy of consideration:

- for the prioritization problems, use the tactic of *deadline monotonic prioritization* (the shorter the deadline the higher the priority),
- for the queuing problems, use the tactic of *priority queues*.

4.6. Rationale for alternatives

In part 2 of the architecture alternative template we recorded the modifications to the analysis model that would result from the application of each alternative. In part 3 we recorded the direct implications and the side effects of each alternative.

Each of the tactics identified will modify the design, presumably to improve it. To ensure that the goal of improvement is met, these modifications need to be reflected in the analysis models. This tight design/analysis loop is necessary to retain engineering control over the process; to ensure that the ramifications of any proposed modification to the architecture are understood and explicitly captured.

For example, the managing of priority queues will result in alleviating potential sources of long latency (by ensuring that high priority requests do not get stuck behind low priority requests in a FIFO queue), but the priority queue may entail additional computational overhead relative to a FIFO queue. Side effects such as this need to be captured in the documentation template, to justify the alternative.

4.7. Impact to business goals and cost/benefit analysis

During the workshop, some alternatives were immediately dismissed as too expensive. Other alternatives required further analysis to determine their cost implications. These activities were performed by the architecture team without the intervention of the evaluation team.

In addition to understanding the costs, we needed to understand the benefits that each alternative provided. Such benefits may be captured in terms of the *utility* that a change is expected to bring to the system (Kazman et al., 2001). The benefit will depend on the quality attribute response goal that can be achieved by the architectural change.

For example, one change might result in missing 1% of the 20 ms deadlines, whereas another might reduce the number of missed deadlines to just .1%. Is this a significant difference? Is the lower percentage of missed deadlines worth the cost of the additional change? To answer these questions, each change needs to be assessed in terms of its benefits, which we quantified in terms of “utility”, following the technique pioneered in the CBAM (Kazman et al., 2001). With this information we were able to calculate an expected ROI (Return on Investment) for each proposed architectural change, and we could then use these ROI values to rank the proposals.

5. An example analysis for variability

As a second example, we will show how we approached one of the variability risk themes, exemplified by the following scenario:

There is a request to integrate a new graphics technology into the platform, the integration is able to be done to level 3 quality within 10 calendar months.

The resellers of the system had long complained that it took an unacceptably long time to integrate new user interface technologies into the system, and that it took too long to bring this integration to an appropriate level of stability. This risk theme, like the one with performance, caused cus-

tomers unhappiness and lost business for the manufacturer. This allowed us to fill out parts 1 and 2 of the analysis template. Part 3 is described next. For this analysis example, rather than describing each step of the method for this example, as we did for performance, we will concentrate on just the analysis framework and the results.

5.1. Detailed analysis of the architecture/stakeholder communication

To reason about variability the following questions need to be answered by the architecture team, with respect to the project in question:

- What needs to vary in the architecture and how often will it happen during a specified timeframe?
- For each variation, what parts (components) of the architecture are affected?
- For each variation point, what other variation points does it depend on?
- For each variation point, what is (are) the variability mechanism(s) to support creation of a new variation? For each variation point, what is (are) the variability mechanism(s) to support selection/removal of existing implementations?
- For each creation variability mechanism, what is the infrastructure (e.g. set of abstract classes, tools, etc.) provided, when and by whom is the mechanism exercised (what skills and/or tools are required)?
- For each selection mechanism what is the infrastructure (e.g. configuration files) provided, when is the mechanism exercised, and by whom is the mechanism exercised (what skills and/or tools are required)?
- For each variability mechanism what is the measure of difficulty to exercise the mechanism? This could be specified in effort, or just a relative measure of complexity on some scale, e.g. 1–5, or Low, Medium, High.

Note that the focus of a variability analysis is more focused on qualitative judgments—identifying potential problem areas and coarse measures of difficulty—rather than on quantitative measures such as one would create for an analysis of performance or availability.

Each of the questions described above were posed to the system architects. An example of their responses, for the new graphics technology scenario, is shown in Table 3.

In this example it was learned that GUI replacement was problematic for the architecture team, and this had been a source of considerable complaint from their integrator/reseller organizations, and a source of lost business.

5.2. Architecture alternatives/analysis model

In their discussion of software product lines, Bachmann and Clements (2005) discuss variability mechanisms and their properties. The variability mechanism used here—component creation—is considered high cost and high risk.

Table 3
Variability decisions

Variation	Variation point in architecture	Variability mechanisms	Infrastructure provided by mechanism	When is the mechanism exercised	By whom is the mechanism exercised	Difficulty
GUI from a customer specification <i>How often:</i> Once per customer project	<i>Component:</i> GUI <i>Depends on:</i> Application, Devices	<i>Creation:</i> Component creation	• Input device interface • Event, Processing/Focus management framework • Presentation, Abstraction Layer	GUI design time	Application developer group	High

In effect, the product line gives little assistance to the integrator/reseller organizations, other than providing a framework into which the newly created component can be inserted.

We then explored some architectural alternatives enumerated in [Bachmann and Clements \(2005\)](#) as a means of ameliorating the problem of unacceptably high cost of tailoring the product line. The alternatives explored included:

- Inheritance,
- component substitution,
- plug-ins,
- generator.

The relative costs and skills required for each of these mechanisms was then enumerated, as shown in [Table 4](#).

5.3. Rationale for alternatives/impact to business goals and cost/benefit analysis

Each of the alternatives presented in [Table 4](#) was then evaluated by the architecture team with respect to its schedule feasibility, set of skills required, impact to business goals, and costs/benefits, as guided by the information contained in the table. In the end, the variability mechanism of Component Substitution—where the architecture team would create a relatively complete set of GUI components from which the reseller/integrator could select and then integrate—was chosen for further investigation. Component Substitution was felt to be feasible for the framework developers to achieve and met one of the organization's key

business goals (that of making the product line framework easy and inexpensive for integrator/resellers to adopt and tailor). Although the cost was significant, the other variability mechanisms available to the architecture team were at least as costly, and most were more costly and more risky. In part this mechanism was less risky because the architecture team felt that they had the in-house skills and domain knowledge necessary to create the set of components.

5.4. Post method activities

We repeated these steps, and created documentation templates addressing each risk theme, enumerating architectural alternatives for each, along with their costs and benefits. At the end of the APTIA the architecture team had enough documented information to make informed business and implementation decisions. After the APTIA they performed two tests: to determine that the theoretical problems identified in the method were indeed problems and to more precisely determine the cost of implementing each architectural alternative. These tests enabled the team to determine the utility of each proposed alternative. Making such architectural decisions had previously posed an enormous challenge to the team, since they had no firm basis on which to guide their architecture decisions.

6. Conclusions and reflections

It may be argued that experienced architects know the kinds of analyses that we have presented here, and do these

Table 4
Relative costs and skills for variability mechanisms (adapted from [Bachmann and Clements \(2005\)](#))

Variability mechanism	Properties for building into core assets	Properties for exercising when building products
Inheritance	<i>Cost:</i> Medium <i>Skills:</i> OO-languages	<i>Stakeholder:</i> Product developers <i>Tools:</i> Compiler <i>Cost:</i> Medium
Component substitution	<i>Cost:</i> Medium <i>Skills:</i> Interface definitions	<i>Stakeholder:</i> Product developer, system administrator <i>Tools:</i> Compiler <i>Cost:</i> Low
Plug-ins	<i>Cost:</i> High <i>Skills:</i> Framework programming	<i>Stakeholder:</i> End user <i>Tools:</i> None <i>Cost:</i> Low
Generator	<i>Cost:</i> High <i>Skills:</i> Generative programming	<i>Stakeholder:</i> System administrator, end user <i>Tools:</i> Generator <i>Cost:</i> Low

analyses regularly as part of their architecting process. While this is true in *some* cases, it is not true in general. Architects are generally the best and the brightest, but they all have their own experiences and limitations. An architect might be an expert on performance, but has little depth of understanding of, say, security or testability. In addition, even if some architects could do this kind of analysis, the point of creating a method is to codify best practices, so that the *average* architect can approximate what the best architects know and do. In the case study presented above, the architecture team, even though they were experienced, intelligent, and well-educated, was aware of many of the risks that we discovered in their architecture, but they did not have the analytic tools and techniques to analyze and improve the architecture on their own.

We began this paper by identifying three core principles of software architecture. Motivated by these principles we have created a number of methods for understanding, eliciting, and analyzing architectural information over the past 10 years. But each of these methods had limitations. This paper suggests that such limitations are arbitrary—they derive from the way that the principles were realized: the specific goals of the method and the constraints under which they operate.

There is, in fact, an enormous number of methods derivable from these principles. We illustrated this claim by first enumerating a number of component techniques and then presenting a method—APTIA—that instantiates these component techniques in a new way, aimed at addressing some limitations of our earlier methods. In our experience, when a method is adopted by others, it is almost never adopted verbatim. Ali Babar et al. (2004) have made this same observation. People create their own tailored version of any method, largely reusing, but also adapting, the techniques embodied within the method.

APTIA has new steps and produces different results than any previous method—in particular it goes deeper into analysis than was possible in our previous methods, and it suggest design alternatives linked to the analysis, rather than simply pointing out potential problems. APTIA was able to guide the architecture team and the analysis team to propose architectural alternatives for a complex system and in the process expose the architecture team to new design principles. And this was all accomplished in a relatively short time period. Each of these alternatives was evaluated with respect to its costs and benefits, and this information gave the architecture team a firm basis for making decisions.

And yet nothing within APTIA is truly new, and that is the exciting message of this paper: APTIA simply reuses and re-combines existing (proven, road-tested) component techniques from our previous wealth of experience with architecture analysis and design:

- quality attribute models,
- design principles in the form of tactics,
- scenario-based quality attribute elicitation and analysis,

- explicit elicitation of the costs and benefits of architectural decisions,
- architectural documentation templates.

By reusing our component techniques we were able to *effectively* reuse our prior experience with other methods. In this way, we believe that architecture analysis and design can be made more agile, reacting to and adapting to environmental conditions without sacrificing quality.

References

- Ali Babar, M., Zhu, L., Jeffrey, R., 2004. A framework for classifying and comparing software architecture evaluation methods. In: Proceedings of 5th Australian Software Engineering Conference, April 2004, pp. 309–319.
- Bachmann, F., Clements, P., 2005. Variability in Software Product Lines (CMU/SEI-2005-TR-012). Software Engineering Institute, Carnegie Mellon.
- Bachmann, F., Bass, L., Klein, M., 2003. Deriving Architectural Tactics: A Step Toward Methodical Architectural Design (CMU/SEI-2003-TR-004). Software Architecture Institute, Carnegie Mellon.
- Barbacci, M., Ellison, R., Lattanze, A., Stafford, J., Weinstock, C., Wood, W., 2003. Quality Attribute Workshops (CMU/SEI-2003-TR-016), third ed. Software Engineering Institute, Carnegie Mellon.
- Bass, L., Clements, P., Kazman, R., 2003. Software Architecture in Practice, second ed. Addison-Wesley, Boston, MA.
- Bengtsson, P.O., Bosch, J., 1998. Scenario-based architecture reengineering. In: Proceedings of the Fifth International Conference on Software Reuse.
- Bengtsson, P.O., Bosch, J., 1999. Architecture level prediction of software maintenance. In: Proceedings Third European Conference on Software Maintenance and Reengineering, March 1999, pp. 139–147.
- Clements, P., Kazman, R., Klein, M., 2002. Evaluating Software Architectures: Methods and Case Studies. Addison-Wesley, Boston, MA.
- Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., Nord, R., Stafford, J., 2003. Documenting Software Architectures: Views and Beyond. Addison-Wesley, Boston, MA.
- Dobrica, L.F., Niemela, E., 2002. A survey on software architecture analysis methods. IEEE Transactions on Software Engineering 28 (7), 638–653.
- Duenas, J., de Oliveira, W., de la Puente, J., 1998. A software architecture evaluation model. In: Proceedings Second International ESPRIT ARES Workshop, February 1998, pp. 148–157.
- Kazman, R., Abowd, G., Bass, L., Webb, M., 1994. SAAM: a method for analyzing the properties of software architectures. In: Proceedings of the 16th International Conference on Software Engineering. Sorrento, Italy, May 16–21, 1994. IEEE Computer Society, Los Alamitos, CA, pp. 81–90.
- Kazman, R., Barbacci, M., Klein, M., Carriere, S., Woods, S., 1999. Experience with performing architecture tradeoff analysis. In: Proceedings of the 21st International Conference on Software Engineering, May 1999, Los Angeles, CA, pp. 54–63.
- Kazman, R., Asundi, J., Klein, M., 2001. Quantifying the costs and benefits of architectural decisions. In: Proceedings of the 23rd International Conference on Software Engineering, May 2001, Toronto, Canada, pp. 297–306.
- Kazman, R., Kruchten, P., Nord, R., Tomayko, J., 2004. Integrating Software-Architecture-Centric Methods into the Rational Unified Process, (CMU/SEI-2004-TR-011). Software Engineering Institute, Carnegie Mellon.

- Kazman, R., Bass, L., Klein, M., Lattanze, A., Northrop, L., 2005. A basis for analyzing software architecture analysis methods. *Software Quality Journal* 13, 329–355.
- Klein, M., Ralya, T., Pollak, B., Obenza, R., Gonzalez Harbour, M., 1993. *A Practitioners Handbook for Real-time Analysis: Guide to Rate Monotonic Analysis for Real-time Systems*. Kluwer Academic Publishers., Boston, MA.
- Lassing, N., Bengtsson, P.O., van Vliet, H., Bosch, J., 2002. Experiences with ALMA: architecture-level modifiability analysis. *Journal of Systems and Software* 61, 47–57.
- Lung, C.-H., Bot, S., Kalaichelvan, K., Kazman, R., 1997. An approach to software architecture analysis for evolution and reusability. In: *Proceedings of CASCON '97*, November 1997.
- Nord, R., Tomayko, J., Wojcik, R., 2004. Integrating Software-Architecture-Centric Methods into Extreme Programming, (CMU/SEI-2004-TN-036). Software Engineering Institute, Carnegie Mellon.
- Shaw, M., 1990. Prospects for an engineering discipline of software. *IEEE Software* 7 (6), 15–24.